Q5) we will use min hepq approach to find kt largest

```
import heapq

class Solution(object):
    def findKthLargest(self, nums, k):

        heap = []

        for num in nums:
            heapq.heappush(heap, num)

            if len(heap) > k:
                heapq.heappop(heap)

        return heap[0]
```

Q1)

```
class Solution(object):
    def productExceptSelf(self, nums):
        n = len(nums)
        ans = [1] * n
        left = 1

        for i in range(n):
            ans[i] = left
            left = left * nums[i]
        right = 1

        for i in range(n - 1, -1, -1):
            ans[i] = ans[i] * right
            right = right * nums[i]

        return ans
```

Q3)

```
class Solution(object):
    def threeSum(self, nums):
        nums.sort()
        ans = []

        for i in range(len(nums) - 2):

            if i > 0 and nums[i] == nums[i - 1]:
                continue

            left = i + 1
            right = len(nums) - 1

            while left < right:
                total = nums[i] + nums[left] + nums[right]

                if total == 0:
                    ans.append([nums[i], nums[left], nums[right]])
                    while left < right and nums[left] == nums[left + 1]:
                        left += 1

                    while left < right and nums[right] == nums[right - 1]:
                        right -= 1

                    left += 1
                    right -= 1

                elif total < 0:
                    left += 1

                else:
                    right -= 1

        return answer
```

Q6)

```
    import heapq

class Solution(object):
    def mergeKLists(self, lists):
        heap = []

        for i in range(len(lists)):
            if lists[i]:
                heapq.heappush(heap, (lists[i].val, i, lists[i]))

        dummy = ListNode(0)
        current = dummy

        while heap:
            value, i, node = heapq.heappop(heap)

            current.next = node
            current = current.next

            if node.next:
                heapq.heappush(
                    heap,
                    (node.next.val, i, node.next)
                )

        return dummy.next
```

Q10)

```
import bisect

class Solution(object):
    def lengthOfLIS(self, nums):
        arr = []

        for num in nums:
            pos = bisect.bisect_left(arr, num)

            if pos == len(arr):
                arr.append(num)
            else:
                arr[pos] = num

        return len(arr)
```

Q7)

```
from collections import deque

class Solution(object):
    def findOrder(self, numCourses, prerequisites):

        graph = [[] for _ in range(numCourses)]
        indegree = [0] * numCourses

        # Build graph
        for course, pre in prerequisites:
            graph[pre].append(course)
            indegree[course] += 1

        queue = deque()

        # Courses with no prerequisite
        for i in range(numCourses):
            if indegree[i] == 0:
                queue.append(i)

        ans = []
```

```
while queue:
    course = queue.popleft()
    ans.append(course)

    # Remove this course as a prerequisite
    for next_course in graph[course]:
        indegree[next_course] -= 1

        if indegree[next_course] == 0:
            queue.append(next_course)

# If all courses are not completed, cycle exists
if len(ans) != numCourses:
    return []

return ans
```